

Learn HTML: Beginner's Guide

Lesson 8: Putting It All Together

Putting It All Together – Study Notes

Key Concepts

1. **Semantic HTML structure** – using `<header>`, `<nav>`, `<section>`, `<article>`, `<aside>`, and `<footer>` to create a logical, accessible page layout.
2. **Responsive design basics** – applying a mobile first CSS layout with media queries, flexible grids, and relative units (`%`, `vh/vw`, `rem`).
3. **Linking resources** – correctly referencing external CSS, JavaScript, fonts, and images; using `<link>` and `<script>` tags with proper `rel`, `href`, `src`, and `defer/async` attributes.
4. **Reusable components** – building navigation menus, hero sections, cards, and footers that can be copied or included across multiple pages.
5. **Accessibility & SEO fundamentals** – adding alt text, ARIA labels, descriptive titles, and meta tags to improve usability and search visibility.

Summary

In this lesson you combine everything you've learned so far—HTML semantics, CSS styling, basic JavaScript, and best practice file organization—to craft a complete multi section web page. You start with a clean HTML skeleton, then layer in a linked stylesheet that defines a mobile first grid, typography, colors, and responsive breakpoints. Navigation is built with an unordered list styled as a horizontal menu that collapses into a “hamburger” icon on small screens. Content sections (hero, features, about, contact) are organized using `<section>` elements, each containing headings, paragraphs, images, and reusable card components. A footer wraps up the page with social media links and copyright information.

The final product demonstrates how each piece works together: semantic markup gives meaning, CSS provides visual layout and responsiveness, and a tiny JavaScript snippet adds interactivity (e.g., toggling the mobile menu). By following accessibility guidelines—alt attributes, proper heading hierarchy, and ARIA roles—the page is usable for all visitors and is better indexed by search engines.

Important Points

- **File structure**

```
project/
% % index.html
% % css/
% % % style.css
% % js/
% % % script.js
```

```
% % assets/
% % images/
% % fonts/
---
```

- **HTML**

- Use `<meta charset="UTF-8">`, `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.

- Keep heading order logical (`<h1>` ! page title, then `<h2>` for sections).

- Wrap navigation links in `<nav>` and list items in `<ul>`.

- **CSS**

- Mobile first: start with a single column layout, then add `@media (min-width: 768px)` for two column, `@media (min-width: 1024px)` for three column.

- Use CSS variables for colors and spacing to keep the design consistent.

- Apply `box-sizing: border-box;` globally.

- **JavaScript** (optional but useful)

- Simple menu toggle:

```
```js
const btn = document.querySelector('.menu-toggle');
const nav = document.querySelector('nav ul');
btn.addEventListener('click', () => nav.classList.toggle('open'));

```

- Add `defer` to the `<script>` tag so the script runs after the HTML is parsed.

- **Accessibility**

- All images need `alt=""` (or descriptive text).

- Add `aria-label` to the menu button (`<button aria-label="Open navigation menu">`).

- Ensure sufficient color contrast (WCAG AA minimum).

- **SEO basics**

- Include `<title>`, `<meta name="description">`, and meaningful heading text.

- Use descriptive file names for images (`hero product.jpg`).

---

## Examples

### Example 1 – Full Page Skeleton (HTML)

```
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Awesome Site</title>
  <meta name="description" content="A demo multi section page built from scratch.">
  <link rel="stylesheet" href="css/style.css">
---
```

```
</head>
<body>
  <header>
    <h1>My Awesome Site</h1>
    <button class="menu-toggle" aria-label="Open navigation menu">&0</button>
    <nav>
      <ul>
        <li><a href="#hero">Home</a></li>
        <li><a href="#features">Features</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>
  <section id="hero" class="hero">
    <h2>Welcome to Our Product</h2>
    <p>Short, compelling tagline goes here.</p>
    <a href="#features" class="btn-primary">Learn More</a>
  </section>
  <section id="features" class="features">
    <h2>Features</h2>
    <div class="cards">
      <article class="card">
        
        <h3>Feature One</h3>
        <p>Brief description of the first feature.</p>
      </article>
      <!-- repeat cards as needed -->
    </div>
  </section>
  <section id="about" class="about">
    <h2>About Us</h2>
    <p>Company story, mission, values...</p>
  </section>
  <section id="contact" class="contact">
    <h2>Contact</h2>
    <form>
      <label for="email">Email</label>
      <input type="email" id="email" required>
      <button type="submit">Send</button>
    </form>
  </section>
```

```

<footer>
  <p>&copy; 2026 My Awesome Site</p>
  <ul class="social">
    <li><a href="#" aria-label="Twitter">Ø=Ü&</a></li>
    <li><a href="#" aria-label="LinkedIn">Ø=Ü¼</a></li>
  </ul>
</footer>
<script src="js/script.js" defer></script>
</body>
</html>
...

```

Explanation: The markup follows a logical flow, uses semantic tags, includes meta data for SEO, and links external CSS/JS files.

Example 2 – Responsive CSS (style.css)

```

```css
/* 1. Variables & global reset */
:root {
 --primary: #2c7be5;
 --secondary: #f8f9fa;
 --text: #212529;
 --max-width: 1200px;
 --gap: 1.5rem;
}
*,
*::before,
*::after { box-sizing: border-box; margin:0; padding:0; }
body { font-family: Arial, sans-serif; color: var(--text); line-height: 1.6; }
/* 2. Layout helpers */
.container { max-width: var(--max-width); margin:auto; padding:0 var(--gap); }
/* 3. Header & navigation */
header { background: var(--primary); color:#fff; padding: var(--gap); }
nav ul { list-style:none; display:none; flex-direction: column; }
nav ul.open { display:flex; }
nav a { color:#fff; text-decoration:none; padding: .5rem 0; }
.menu-toggle { background:none; border:none; color:#fff; font-size:1.5rem; cursor:pointer; }
/* 4. Hero section */
.hero { background: url('../assets/images/hero.jpg') center/cover no-repeat;
 color:#fff; text-align:center; padding:4rem var(--gap); }
.btn-primary { background: var(--secondary); color:var(--primary);
 padding:.75rem 1.5rem; border:none; cursor:pointer; }

```

```

/* 5. Feature cards – mobile first (single column) */
.features .cards { display:grid; gap: var(--gap); }
.card { background:#fff; padding:var(--gap); border-radius:.5rem; box-shadow:0 2px 5px rgba(0,0,0,.1); }
.card img { width:100%; height:auto; border-radius:.5rem .5rem 0 0; }
/* 6. Responsive breakpoints */
@media (min-width: 768px) {
 nav ul { display:flex; flex-direction:row; gap:1rem; }
 .menu-toggle { display:none; }
 .features .cards { grid-template-columns: repeat(2,1fr); }
}
@media (min-width: 1024px) {
 .features .cards { grid-template-columns: repeat(3,1fr); }
 .hero { padding:6rem var(--gap); }
}
/* 7. Footer */
footer { background:#333; color:#fff; text-align:center; padding:var(--gap); }
footer .social { display:flex; justify-content:center; gap:.5rem; }
footer .social a { color:#fff; font-size:1.2rem; }
...

```

**\*Explanation\*:** The stylesheet starts with a mobile first layout, then adds two breakpoints (768 /px and 1024 /px) to expand the navigation and card grid. CSS variables keep colors and spacing consistent, and the ``.menu-toggle`` button only appears on small screens.

---

## Quick Review

1. **\*\*Why should you start a CSS file with a mobile first approach?\*\***
2. **\*\*Name three semantic HTML elements you would use to separate a page into distinct sections.\*\***
3. **\*\*What attribute do you add to a `<script>` tag to ensure it runs after the HTML is parsed?\*\***
4. **\*\*How can you make a navigation menu accessible for screen reader users?\*\***
5. **\*\*What CSS property is used to create a flexible grid for the feature cards?\*\***

---

## Further Reading

- **\*\*Advanced CSS Layout\*\*** – Flexbox vs. CSS Grid, named lines, and sub grid.
- **\*\*Progressive Enhancement\*\*** – Adding JavaScript features without breaking the base HTML/CSS.
- **\*\*Web Accessibility (WCAG 2.2)\*\*** – Detailed ARIA roles, focus management, and color contrast testing.
- **\*\*SEO Best Practices\*\*** – Structured data (JSON LD), Open Graph tags, and sitemap creation.
- **\*\*Build Tools\*\*** – Introduction to npm scripts, PostCSS, and simple bundlers like Vite for larger projects.

